

Backend — Financeiro, Fluxo de Caixa e Produção Mensal

Esta seção documenta os endpoints do backend (`C:\Dashboard\backend\main.py` , FastAPI) responsáveis pelos cálculos financeiros e de produção: faturamento, recebimentos, fluxo de caixa, metas, valor por hora (médico/consultório) e o motor de projeção da Produção Mensal.

Helpers Compartilhados

`get_conn()` / `query(sql, params)`

`get_conn()` abre uma conexão pyodbc com o SQL Server do sistema Píxeon (`DATABASE=SMART` , servidor `192.168.1.9,1433`). `query()` executa um SQL, fecha a conexão e retorna uma lista de dicts (uma por linha, chaves = nomes das colunas). É a função usada por quase todos os endpoints de leitura. Não há commit automático — é só para SELECT.

Existe também `get_conn_hml()` , que conecta no banco de **homologação** (`smart_hml`), reservado para escrita de registros clínicos (RCL) ainda não validados — não usado nos endpoints financeiros.

`periodo_datas(periodo: str) -> (inicio, fim)`

Converte uma string de período recebida via query param em um intervalo de datas (`YYYY-MM-DD` , `YYYY-MM-DD`):

Valor de <code>periodo</code>	Intervalo resultante
"hoje"	Só o dia de hoje
"7d"	Últimos 7 dias corridos
"30d"	Mês atual completo (dia 1 até hoje)
"90d"	Últimos 3 meses completos
"ano"	Ano corrente (1/jan até hoje)
"mes:YYYY-MM"	Mês específico (dia 1 até o último dia; se for o mês atual, vai só até hoje)
"custom:YYYY-MM-DD:YYYY-MM-DD"	Intervalo arbitrário informado pelo usuário
qualquer outro valor	Cai no dicionário <code>PERIODOS = { "7d":7, "30d":30, "90d":90 }</code> , tratando como "últimos N dias"; default 30 dias se não reconhecido

Outros helpers usados nesta área

- `periodo_anterior(inicio, fim)` : retorna o período imediatamente anterior, com a mesma duração — usado para comparações "vs. período anterior".
- `var_pct(atual, anterior)` : variação percentual entre dois valores; retorna `None` se não houver base de comparação (`anterior = 0`).

- `filtro_setores_sql(setores, alias_smm)` : monta uma cláusula `WHERE ... AND smm.SMM_STR IN (...)` a partir de uma string de setores separados por vírgula.
- `_load_metas()` / `_save_metas()` : metas por módulo são persistidas em um arquivo local `metas_config.json` (não no banco SMART, que pertence ao sistema Píxeon) — mesmo padrão usado para configuração do WhatsApp (`whatsapp_config.json`).

Fórmula de valor líquido (usada em quase todo endpoint financeiro)

```
valor_liquido = SMM_VLR - ISNULL(SMM_VLR_DESCONTO,0) - ISNULL(SMM_VLR_COPARTIC,0) + ISNULL(SMM_AJUSTE_VLR,0)
```

Ou seja: valor bruto do item de serviço, menos desconto concedido, menos coparticipação do convênio, mais qualquer ajuste manual. Essa expressão aparece repetida literalmente em dezenas de queries (não foi extraída para uma função por ser SQL embutido em cada query).

Classificação de atendimento usada nos agrupamentos "Ocupacional x Assistencial": - **Assistencial** = `osm_atend = 'ASS'` - **Medicina Ocupacional** = `osm_atend IN ('ADM','PER','DEM','RTB','MDF','MOC')` (Admissional, Periódico, Demissional, Retorno ao Trabalho, Mudança de Função, e um genérico MOC)

Filtro de "produção válida" recorrente: `smm.SMM_SFAT IN ('A','F','P')` — Aberto, Faturado, Pendente (exclui itens cancelados/estornados, cujo código não está nesse conjunto).

/api/financeiro/* — Faturamento e Recebimentos

GET /api/financeiro/resumo

Params: `periodo`, `atend` (código de tipo de atendimento), `setores`. Soma o valor líquido (fórmula acima) de `smm` join `osm` no período, com breakdown por tipo de atendimento (assistencial, cada subtipo de ocupacional, emergência, cirurgia) e por status de faturamento (`val_faturado`, `val_aberto`, `val_pendente`). Também calcula `ticket_medio = faturamento / total_os` (OS distintas).

GET /api/financeiro/receita-mensal

Params: `periodo` (não usado para filtrar — sempre olha os últimos 6 meses fixos via `DATEADD(month,-6,GETDATE())`), `atend`. Receita mensal agrupada por `FORMAT(osm_dthr,'yyyy-MM')`, com breakdown faturado/aberto/pendente.

GET /api/financeiro/por-convenio

Params: `periodo`, `atend`, `setores`. Receita por convênio, via `osm.osm_cnv` (FK principal de convênio na OS — comentário no código explica que `smm.SMM_CNV_COD` fica vazio em muitos registros, por isso não é usado aqui).

GET /api/financeiro/por-tipo-convenio

Params: `periodo`, `atend`. Receita agrupada por `cnv.cnv_tipo` (AM=Ambulatorial, HP=Hospitalar, AH=Ambul/Hosp, MC=Medicina Ocupacional), via `smm.SMM_CNV_COD` (aqui sim, com `cnv.cnv_stat='A'` — convênio ativo).

GET /api/financeiro/particular

Params: `periodo`, `atend`. Soma tudo cujo `cnv.cnv_nome` contenha "PARTICULAR" (case-insensitive, `LIKE '%PARTICULAR%'`), com breakdown por convênio particular encontrado (às vezes há mais de um cadastro de convênio particular).

GET /api/financeiro/recebimentos

Params: `periodo` . Recebimento **real** de caixa via tabela `mte` (Movimento de Tesouraria) — diferente de faturamento (que é o valor gerado pelo serviço, não necessariamente recebido). Filtra `mte_del_logica <> 'S'` e `mte_estorno <> 'S'` (cancelados/estornados sempre excluídos). Calcula `liquido = valor - desconto + juros` .

GET /api/financeiro/recebimentos-por-dia

Params: `periodo` . Mesma fonte (`mte`), agrupado por dia — usado para gráfico de linha.

GET /api/financeiro/comparativo

Params: `periodo` (não usado para filtro — sempre últimos 6 meses fixos). Uma das visões financeiras centrais: junta três fontes por mês: - **Faturado e Em Aberto**: tabela `fat` (`fat_val` , `fat_sld` = saldo em aberto), agrupado por `FORMAT(fat_demi, 'yyyy-MM')` . - **Recebido (caixa)**: `mte` , mesmo filtro de exclusão de cancelados/estornos. - **Pago (despesas)**: `IPG` com `IPG_STATUS='R'` (realizado/pago), por `IPG_DT_PGTO` .

Resultado por mês: `faturado`, `em_aberto`, `recebido_fat` (`fat_val - fat_sld`), `recebido_caixa` (`mte`), `pago_despesas` (`IPG`) .

GET /api/financeiro/ocupacional-vs-assistencial

Params: `meses` (default 24). Produção mensal lado a lado (Ocupacional x Assistencial) ao longo de N meses, para visualizar cruzamentos de tendência.

GET /api/financeiro/faturamento-anual

Params: `anos` (default 3). Para cada um dos últimos N anos, soma o valor líquido mês a mês (meses futuros do ano corrente ficam como `null`). Usado para comparativo ano a ano.

Fluxo de Caixa (/api/financeiro/fluxo-caixa/*)

Bloco introduzido para dar ao módulo Produção uma visão de caixa bidirecional (entradas x saídas reais), não só receita. Fontes: -

Entradas: `mte` (recebimento real). - **Saídas**: `CPG` (cabeçalho da despesa) + `IPG` (parcelas). `IPG_STATUS` : `P` =pendente, `R` =pago/realizado, `C` =cancelado, `A` =aberto. - **Categoria de despesa**: `CCT` (plano de contas), via `CPG.CPG_CCT_COD_PASSIVO` . - **Fornecedor**: `PSV` (pessoa), via `CPG.CPG_PSV_COD` , com fallback para o texto livre `CPG_CREDOR` quando não há fornecedor cadastrado vinculado.

Nota registrada no código: CPG/IPG têm uso histórico concentrado em 2018-2020 (séries 117-119) — a clínica hoje controla despesa majoritariamente fora do sistema. Dados antigos ainda aparecem no relatório; a partir da introdução do `POST /api/financeiro/despesas` , novos lançamentos feitos pelo próprio Dashboard passam a alimentar essas tabelas.

GET /api/financeiro/fluxo-caixa/resumo

Params: `periodo` . - `entradas` = `SUM(mte.mte_valor)` no período (mesmos filtros de exclusão de cancelado/estorno). - `saídas` = `SUM(IPG_VALOR)` onde `IPG_STATUS='R'` e `IPG_DT_PGTO` no período. - `saldo` = `entradas - saidas` . - `a_receber_30d` = soma de `fat.fat_sld` (saldo em aberto) cujo vencimento (`fat_venc`) cai nos **próximos 30 dias** — **não** é o total de `fat_sld` em aberto (esse total ultrapassa R\$75 milhões, pois acumula faturas desde 2017, a maioria vencida há anos sem baixa/glosa processada — por isso é isolado à parte). - `em_atraso_valor` / `em_atraso_qtd` = soma/contagem de `fat_sld > 0` com `fat_venc < hoje` —

contexto de cobrança, propositalmente **não** entra no saldo projetado. - `a_pagar_30d` = soma de `IPG_VALOR` com `IPG_STATUS='P'` e vencimento (`IPG_DT_VCTO`) nos próximos 30 dias. - `saldo_projetado_30d` = `saldo` + `a_receber_30d` - `a_pagar_30d` .

GET /api/financeiro/fluxo-caixa/diario

Params: `periodo` . Série diária de entrada (`mte`) e saída (`IPG` pago) no período, com `saldo_acumulado` calculado incrementalmente dia a dia (soma corrida de entrada-saída).

GET /api/financeiro/fluxo-caixa/projecao

Params: `dias` (default 30). Para os próximos N dias a partir de hoje: `a_receber` (`fat_sld>0` por `fat_venc`) e `a_pagar` (`IPG_STATUS='P'` por `IPG_DT_VCTO`), com `saldo_projetado` acumulado dia a dia.

GET /api/financeiro/fluxo-caixa/categorias

Params: `periodo` . Despesas pagas (`IPG_STATUS='R'`) agrupadas por categoria (`CCT.CCT_DESCR` , via `CPG_CCT_COD_PASSIVO`). Categoria `NULL` vira "Sem categoria".

GET /api/financeiro/fluxo-caixa/fornecedores

Params: `periodo` . Top 15 fornecedores por valor pago, nome resolvido via `PSV.PSV_NOME` com fallback para `CPG.CPG_CREDOR` (texto livre) quando não há PSV vinculado.

GET /api/financeiro/centros-custo

Sem params. Lista todos os registros de `CCT` (código + descrição) — usado para popular o seletor de centro de custo no formulário de nova despesa.

GET /api/financeiro/fornecedores/busca

Params: `q` (busca, mínimo 2 caracteres). Busca em `PSV` filtrando `PSV_TIPO='M'` (tipo confirmado, via investigação prévia, como o valor usado por 100% dos fornecedores já vinculados a despesas reais) e nome `LIKE '%q%'` . Retorna até 20 resultados (código, nome, CPF).

POST /api/financeiro/despesas — Lançamento de nova despesa

Único endpoint de **escrita** nesta área — grava diretamente nas tabelas de produção `CPG` / `IPG` .

Corpo (`DespesaRequest`): `fornecedor_nome` , `psv_cod` (opcional), `fis_jur` ("F" ou "J"), `cic_rg` (CPF/CNPJ, opcional), `cct_cod` (opcional), `descricao` , `valor_total` , `parcelas` (default 1), `data_primeira_parcela` .

Validações: `fis_jur` deve ser F/J; `valor_total>0` ; `parcelas>=1` ; `fornecedor_nome` e `descricao` obrigatórios; `data_primeira_parcela` no formato `YYYY-MM-DD` .

Série exclusiva: usa `CPG_SERIE = 200` , escolhida por não colidir com nenhuma série usada pelo aplicativo desktop da Píxeon (séries 117-120 = uso histórico 2018-2020; séries 123-126 = reembolsos automáticos a pacientes, `CPG_TIPO_COMPROMISSO='U'`). Isso evita qualquer mistura semântica com lançamentos gerados pelo sistema.

Defaults confirmados via investigação da distribuição real de lançamentos existentes: - `CPG_EMP_COD = 0` ("Não especificado") - `CPG_GCC_COD = '1'` (ICDS - Clínica de Especialidades, marcado como `GCC_DEFAULT`) - `CPG_TIPO_COMPROMISSO = 'N'` (compromisso normal/genérico — mesmo valor usado em lançamentos reais de fornecedor, distinto de 'I'=retenção de imposto e 'U'=reembolso)

Cálculo de parcelas: `valor_parcela = round(valor_total / parcelas, 2)` ; a última parcela absorve a diferença de arredondamento (`diff = valor_total - soma(parcelas)`), para o total bater exatamente com `valor_total` . Vencimentos: `data_primeira_parcela + 30*i dias` para a parcela `i` (não usa "mesmo dia do mês seguinte", e sim +30 dias corridos). Todas as parcelas entram com `IPG_STATUS='P'` (pendente).

Transação: usa `WITH (UPDLOCK, HOLDLOCK)` no cálculo do próximo `CPG_NUM` (`MAX(CPG_NUM)+1` dentro da série 200) para evitar colisão com lançamentos concorrentes (inclusive do próprio aplicativo desktop da Píxeon rodando ao mesmo tempo). **Importante (comentário explícito no código):** não usar `BEGIN TRANSACTION` explícito — o `autocommit=False` padrão do pyodbc já mantém uma transação implícita; um `BEGIN TRANSACTION` adicional aninha uma segunda transação que o `commit()` do pyodbc não fecha por completo, fazendo o INSERT ser revertido silenciosamente ao fechar a conexão (bug real encontrado e corrigido durante o desenvolvimento).

| Valor por Hora (`/api/agenda/consultorios/valor-hora` e `/api/agenda/medicos/valor-hora`)

Ambos calculam **quanto se gerou de valor por hora efetivamente ocupada**, usando dados de agendamento (`agm`), não de faturamento (`smm` / `fat`) — porque `agm` é a única fonte com horário de início/fim (`agm_hini` / `agm_hfim`) por sala/médico.

Regra comum: só considera `agm_stat = 'E'` (Executado — o único status em que o horário realmente ocorreu e o valor foi de fato gerado), exige `agm_hfim > agm_hini` , e só entra no ranking se a soma de horas ocupadas no período for **>= 60 minutos** (filtro para não deixar amostras minúsculas distorcerem o valor/hora).

```
horas_ocupadas = SUM(DATEDIFF(MINUTE, agm_hini, agm_hfim)) / 60.0
valor_hora     = valor_total / horas_ocupadas
```

GET `/api/agenda/consultorios/valor-hora`

Params: `periodo` . Agrupa por sala/consultório (`LOC` , join `loc.loc_cod = agm.agm_loc`). Observação registrada no código: `agm_valor` é o valor do agendamento em si — não é o mesmo dado de recebimento real do módulo `/api/financeiro` , mas é a melhor granularidade disponível por sala, já que `smm` / `fat` não guardam local de forma confiável.

GET `/api/agenda/medicos/valor-hora`

Params: `periodo` . Agrupa por médico, mas com uma correção importante: **atribuição por médico executor, não pelo dono do horário agendado**. Em cerca de 3% dos atendimentos, o médico que de fato executou um procedimento (`smm.SMM_MED` — ex: um exame feito por outro especialista dentro da mesma consulta) é diferente de quem estava na agenda (`agm.AGM_MED`).

A query resolve isso com uma CTE (`executor_visita`) que, para cada visita, busca via subquery correlacionada o `SMM_MED` do item de **maior valor** (`SMM_VLR DESC`) daquela mesma OS (join por `AGM_OSM_SERIE / AGM_OSM_NUM = SMM_OSM_SERIE / SMM_OSM`), caindo para `AGM_MED` quando não há nenhum `SMM_MED` vinculado. Duração e valor continuam vindo do agendamento (`agm`) — só o médico creditado muda.

| Metas (`/api/metast/*`)

Metas por módulo são armazenadas em arquivo local `metas_config.json` (fora do banco SMART, que pertence ao sistema Píxeon).

GET `/api/metast`

Retorna todo o objeto configurado: `{ modulo: { meta_mensal, meta_diaria, meta_sabado } }` .

PUT /api/metast/{modulo}

Corpo: `{ meta_mensal?, meta_diaria?, meta_sabado? }`. Atualiza (parcialmente, preservando valores não enviados) a meta daquele módulo.

DELETE /api/metast/{modulo}

Remove a configuração de meta daquele módulo.

Motor de Projeção — Produção Mensal

GET /api/financeiro/producao-mensal

Params: `ano`, `mes` (default: mês atual), `meta_diaria`, `meta_mensal_fixa` (default `1.200.000,00`), `meta_sabado`.

Este é o cálculo mais elaborado da plataforma — projeta se o mês vai bater a meta.

- Produção diária:** soma o valor líquido do mês inteiro, dia a dia, com breakdown Ocupacional x Assistencial (mesmos códigos `osm_atend` de sempre).
- Feriados de Parauapebas/PA:** função interna `feriados_ano(ano)` calcula um conjunto de datas de feriado — nacionais fixos, Páscoa (via algoritmo de Gauss, com Sexta-feira Santa, Carnaval segunda/terça-feira como opcionais, e Corpus Christi derivados dela), estadual (15/08 — Adesão do Pará à Independência) e municipal (27/05 — aniversário de Parauapebas).
- Peso de cada dia do mês** (`_peso_dia`): domingo ou feriado = peso 0 (não conta como dia útil); sábado = peso proporcional `meta_sabado / meta_diaria` (pondera o sábado como "vale menos" que um dia de semana cheio); qualquer outro dia = peso 1.
- Dias úteis do mês** = soma dos pesos de todos os dias do mês. **Dias restantes** = mesma soma, só para os dias ainda não vividos (se for o mês corrente).
- Meta diária:** se não informada, é derivada de `meta_mensal_fixa / dias uteis_mes`.
- Meta do mês:** usa `meta_mensal_fixa` diretamente se informada; senão `meta_diaria * dias uteis_mes`.
- Projeção:** `total_geral_ate_agora + (media_diaria_ate_agora * dias_restantes)` — projeção linear ingênua baseada na média diária observada até o momento.
- Diferença:** `meta_mes - total_geral` (positivo = falta bater, negativo = já superou).

As configurações de meta usadas como default (`meta_diaria`, `meta_sabado`) vêm de `_load_metast()["producao"]` quando não informadas explicitamente via query param.

GET /api/financeiro/recordes

Sem params obrigatórios. Varre ano a ano (2017 até o ano corrente) somando valor líquido por dia, identificando o melhor dia, melhor mês e melhor ano histórico de faturamento. Cada recorde vem com um breakdown "por recepção" (RDI/ROC/RCN/RCI, mesmo agrupamento usado na mensagem de fechamento do WhatsApp).

Cacheado em memória por 1 hora (`_RECORDES_CACHE`) — comentário no código explica que a consulta ano-a-ano leva ~30s no total (cada ano isolado com filtro de data, para o SQL Server usar o índice de `osm_dthr`; testado e confirmado que uma única query sem filtro de data no histórico inteiro trava por mais de 90s, provavelmente por plano de execução ruim). Como recorde só pode ser batido daqui pra frente (nunca retroativamente), o cache de 1h é seguro.

GET /api/financeiro/producao-diaria-recepcao

Params: `ano`, `mes`. Produção líquida diária, aberta por ponto de recepção (`osm_str` — RDI, ROC, RCN, RCI), com o item especial de que **PSI soma dentro de RCN** (mesmo critério usado no Painel de Senhas).

GET /api/financeiro/producao-mensal/profissionais

Params: `ano` , `mes` . Produção por profissional, combinando duas óticas: - **Executado**: serviços onde o profissional é `COALESCE(smm.SMM_MED, osm.osm_mreq)` (executor real, com fallback para o médico requisitante). - **Solicitado**: serviços que o profissional pediu (`osm.osm_mreq`) mas que foram executados por **outro** médico (`smm.SMM_MED <> osm.osm_mreq`) — evita contar duas vezes o que ele mesmo executou.

Cada serviço é classificado em Consulta / Exame / Imagem / Procedimento / Outros via análise de palavras-chave no nome do serviço (`classificar()`), função local: procura substrings como "CONSULTA", "EXAME", "RAIO", "ULTRASSOM", etc. no nome em maiúsculas).

GET /api/financeiro/producao-mensal/profissional-servicos

Params: `profissional` (nome/apelido exato), `ano` , `mes` . Detalhamento de serviços executados e solicitados por aquele profissional específico, mesma lógica de separação executado/solicitado do endpoint acima.